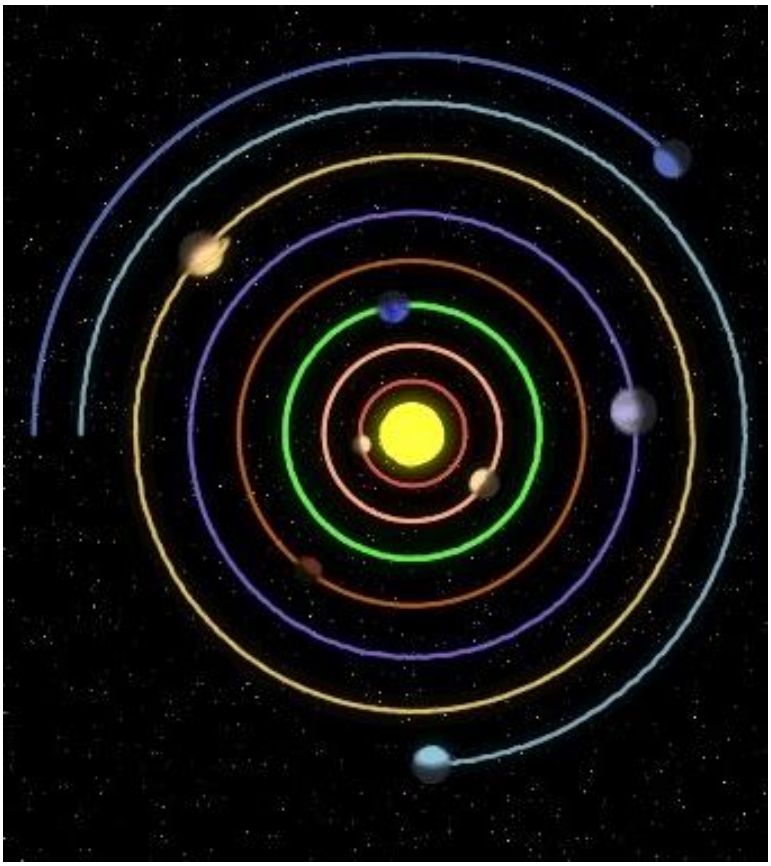


Manual de Usuario: Simulación de Sistema Solar

Proyecto 4

Equipo 2:

-Cervantes González Gabriela Naomi
-García García María Fernanda
-Hernandez Sosol María Fernanda
-Zopillaxtle Sánchez Perla Sofía

Tabla de contenido

Introducción.....2

Requisitos técnicos para ejecutar el programa.....3

Instrucciones de uso.....4

Conclusiones.....7

Código fuente.....8

Introducción

Objetivo del proyecto

Desarrollar un programa que simule el movimiento orbital de planetas en un sistema solar simplificado utilizando matrices de transformación lineal, específicamente matrices de rotación y escalado, para calcular y visualizar trayectorias elípticas en tiempo real. El proyecto permitirá a los usuarios ajustar parámetros como el radio orbital y la velocidad angular para personalizar la simulación.

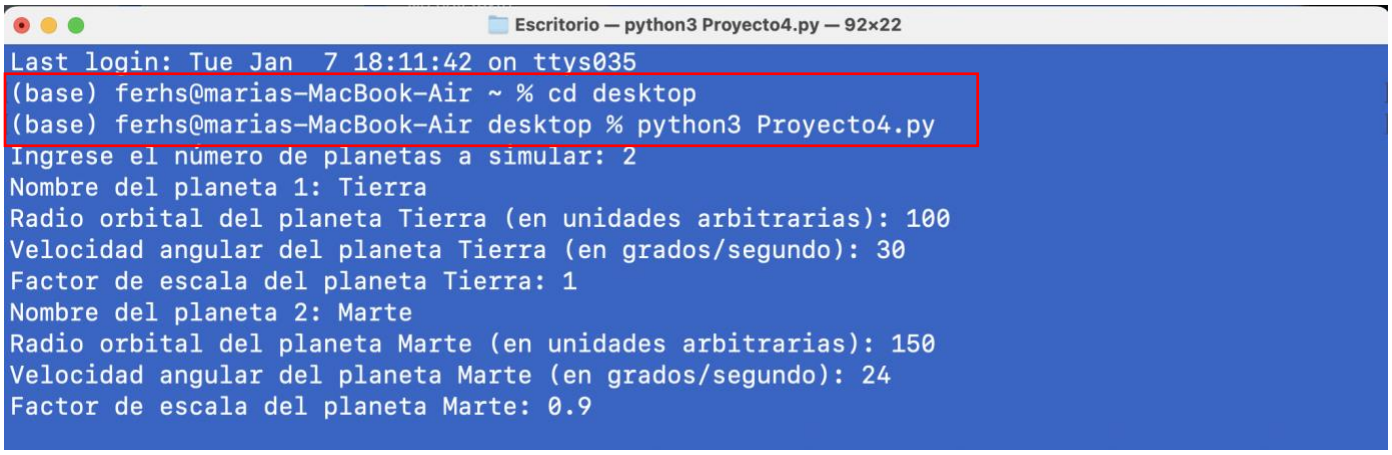
Requisitos técnicos para ejecutar el programa

- Python instalado
- Librería numpy
- Librería matplotlib

Instrucciones de uso

Ejecución del Programa

- Asegúrate de tener Python instalado en tu sistema (versión 3.7 o superior).
- Verifica que las librerías necesarias estén instaladas:
  - numpy
  - matplotlib
- Abre una terminal o consola y navega al directorio donde guardaste el archivo.
- Ejecuta el programa con el siguiente comando: python3 Proyecto4.py



Introducción de Datos

Cuando ejecutes el programa, se te solicitará que ingreses los parámetros de los planetas:

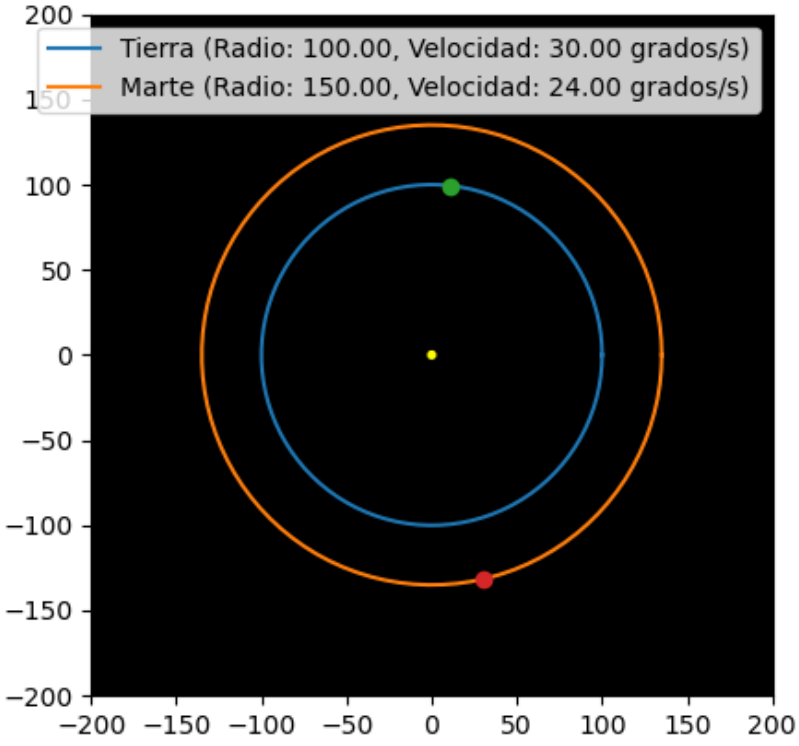
- Ingresa el número de planetas que deseas simular:
- Para cada planeta, introduce los siguientes datos:
  - **Nombre del planeta**: Elige un nombre descriptivo (por ejemplo, "Tierra").
  - **Radio orbital**: La distancia del planeta al sol (en unidades arbitrarias).
  - **Velocidad angular**: Velocidad de movimiento orbital (en grados por segundo).

```
Escritorio — python3 Proyecto4.py — 92×22
Last login: Tue Jan  7 18:11:42 on ttys035
(base) ferhs@marias-MacBook-Air ~ % cd desktop
(base) ferhs@marias-MacBook-Air desktop % python3 Proyecto4.py
Ingrese el número de planetas a simular: 2
Nombre del planeta 1: Tierra
Radio orbital del planeta Tierra (en unidades arbitrarias): 100
Velocidad angular del planeta Tierra (en grados/segundo): 30
Factor de escala del planeta Tierra: 1
Nombre del planeta 2: Marte
Radio orbital del planeta Marte (en unidades arbitrarias): 150
Velocidad angular del planeta Marte (en grados/segundo): 24
Factor de escala del planeta Marte: 0.9
```

Visualización del Sistema Solar

Una vez que ingreses los datos, el programa generará un **gráfico interactivo:**

- El sol en el centro (en color amarillo).
- Las órbitas de los planetas como trayectorias circulares o elípticas.
- Los planetas moviéndose en tiempo real a lo largo de sus órbitas.
- Parámetros de los planetas



Finalización del Programa

- Para cerrar la simulación, cierra la ventana del gráfico.
- Si deseas reiniciar, simplemente vuelve a ejecutar el programa.

Consejos Adicionales

- Introduce valores razonables para el radio orbital y la velocidad angular para evitar planetas con movimientos extremadamente lentos o rápidos.
- Ajusta el factor de escala para planetas más lejanos y hacerlos visibles dentro de los límites del gráfico.

Conclusiones

El programa de simulación del sistema solar proporciona una herramienta educativa e interactiva que permite a los usuarios explorar conceptos relacionados con las órbitas planetarias, como el movimiento circular, la velocidad angular y el escalado de distancias. Los puntos clave son:

1. **Interactividad**: Los usuarios pueden personalizar el número de planetas y sus parámetros, lo que permite experimentar con diferentes configuraciones orbitales.
2. **Visualización Intuitiva**: La animación y el gráfico generado ayudan a comprender mejor el movimiento de los planetas en un entorno espacial simplificado.
3. **Flexibilidad**: El uso de Python y librerías como matplotlib hace que el programa sea fácil de modificar y ampliar para incluir características adicionales, como excentricidad orbital o efectos gravitacionales.
4. **Educación Científica**: Es una excelente herramienta para introducir conceptos de física y matemáticas relacionados con las órbitas y la cinemática.

Este programa puede servir como base para simulaciones más complejas en el futuro, integrando aspectos como trayectorias elípticas reales, gráficas de energía orbital o incluso modelos tridimensionales del sistema solar.

Código fuente

```
#Proyecto 4: Simulación de Órbitas Planetarias en 2D

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation

class Planeta:
    def __init__(self, nombre, radio_orbital, velocidad_angular,
factor_escala):
        self.nombre = nombre
        self.radio_orbital = radio_orbital
        self.velocidad_angular = np.deg2rad(velocidad_angular)  #
Convertimos a radianes
        self.factor_escala = factor_escala
        self.angulo = 0  # Posición inicial en la órbita

    def matriz_rotacion(self, angulo):
        #Genera la matriz de rotación para un ángulo dado.
        return np.array([[np.cos(angulo), -np.sin(angulo)],
                         [np.sin(angulo),  np.cos(angulo)]])

    def matriz_escalado(self, factor_escala):
        #Genera la matriz de escalado para un factor dado.
        return np.array([[factor_escala, 0],
                         [0, factor_escala]])

    def obtener_posicion(self):
        #Calcula la posición actual del planeta en el espacio 2D usando
matrices.
        vector_posicion = np.array([self.radio_orbital, 0])  # Posición
inicial en el eje X
        rotacion = self.matriz_rotacion(self.angulo)
        escalado = self.matriz_escalado(self.factor_escala)
        posicion = escalado @ (rotacion @ vector_posicion)
        return posicion[0], posicion[1]

    def actualizar_posicion(self, dt):
        #Actualiza la posición angular del planeta.
        self.angulo += self.velocidad_angular * dt
        self.angulo %= 2 * np.pi  # Aseguramos que el ángulo esté en el
rango [0, 2pi)

# Configuraciones iniciales del sistema solar
planetas = []

def configurar_planetas():
    num_planetas = int(input("Ingrese el número de planetas a simular:
"))
    for i in range(num_planetas):
```

```
        nombre = input(f"Nombre del planeta {i + 1}: ")
        radio_orbital = float(input(f"Radio orbital del planeta {nombre}
(en unidades arbitrarias): "))
        velocidad_angular = float(input(f"Velocidad angular del planeta
{nombre} (en grados/segundo): "))
        factor_escala = float(input(f"Factor de escala del planeta
{nombre}: "))
        planetas.append(Planeta(nombre, radio_orbital, velocidad_angular,
factor_escala))

# Configurar y ejecutar la simulación
configurar_planetas()

# Mostrar los parámetros de cada planeta en consola
print("\nParámetros de los planetas:")
for planeta in planetas:
    print(f"Nombre: {planeta.nombre}, Radio orbital:
{planeta.radio_orbital}, Velocidad angular:
{np.rad2deg(planeta.velocidad_angular)} grados/segundo, Factor de escala:
{planeta.factor_escala}")

# Visualización
fig, ax = plt.subplots()
ax.set_aspect('equal', 'box')
ax.set_xlim(-200, 200)
ax.set_ylim(-200, 200)
ax.set_facecolor('black')  # Establecer fondo negro
sol = plt.Circle((0, 0), 2, color='yellow')
ax.add_artist(sol)
trayectorias = [ax.plot([], [], label=f"{planeta.nombre} (Radio:
{planeta.radio_orbital:.2f}, Velocidad:
{np.rad2deg(planeta.velocidad_angular):.2f} grados/s)")[0] for planeta in
planetas]
puntos = [ax.plot([], [], 'o')[0] for _ in planetas]

# Función de inicialización
def init():
    for trayectoria in trayectorias:
        trayectoria.set_data([], [])
    for punto in puntos:
        punto.set_data([], [])
    return trayectorias + puntos

# Función de actualización para la animación
def update(frame):
    for i, planeta in enumerate(planetas):
        planeta.actualizar_posicion(0.1)
        x, y = planeta.obtener_posicion()
        trayectorias[i].set_data(planeta.radio_orbital *
planeta.factor_escala * np.cos(np.linspace(0, 2 * np.pi, 100)),
                                 planeta.radio_orbital *
planeta.factor_escala * np.sin(np.linspace(0, 2 * np.pi, 100)))
        puntos[i].set_data(x, y)
```

```
    return trayectorias + puntos

ani = FuncAnimation(fig, update, frames=np.arange(0, 100, 0.1),
init_func=init, blit=True, interval=50)
ax.legend()
plt.show()
```